Document number	P2740R0
Date	2022-12-12
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

Simpler implicit dangling resolution

Table of contents

Abstract

The `Simpler implicit move` proposal [1] provided some dangling relief to the `C++` language. However, this relief was a consequence of fixing `implicit move`. Consequently, it did not clearly address similarly related types of dangling. This paper asks that similar fixes apply to related scenarios in order to ease more dangling in the language. This by no means fixes all dangling in general but it does compliment like minded proposals that address other contributing factors.

Motivation

There are multiple resolutions to dangling in the `C++` language.

1. Produce an error
   - `Simpler implicit move` [1:1]
   - **This proposal**

2. Fix with block/variable scoping
    - `Fix the range-based for loop, Rev2` [2]
    - `Get Fix of Broken Range-based for Loop Finally Done` [3]
3. Fix by making the instance global

All are valid resolutions and individually are better than the others, given the scenario. This proposal is focused on the first option, which is to produce errors when code is clearly wrong.

We first need to examine the dangling fixes provided by the `Simpler implicit move` [1:2] proposal. These fixes helps to mitigate returning references to locals which is always wrong and as such should be errors in the programming language.

> **C++ Core Guidelines**
> **F.43: Never (directly or indirectly) return a pointer or a reference to a local object** [4]
>
> ***Reason*** *To avoid the crashes and data corruption that can result from the use of such a dangling pointer.* [4:1]
>
> …
>
> ***Note*** *This applies only to non-static local variables.* [4:2]

The resolution in this **first example** addresses **directly** returning a reference to a local.

```
int& h(bool b, int i) {
  static int s;
  if (b) {
    return s;  // OK
  } else {
    return i;  // error: i is an xvalue
  }
}
```

The resolution in this **second example** addresses one additional level of **indirectly** returning a `std::reference_wrapper` to a local.

```
std::reference_wrapper<Widget> fifteen() {
    Widget w;
    return w;  // OK until CWG1579; OK after LWG2993. Proposed: ill-formed
}
```

Both fixes are welcomed but much more can be gained at minimal cost.

Motivating Examples

Any given function, already knows whether it returns a pointer or a reference. It knows whether any instances within the function are locals or globals. It also knows the lifetimes of these instances and the pointers and references that refers to them. In other words, a compiler does not need to go outside of the function in question for this information with the exception of globals. **Direct** pointers/references are easy but **indirect** requires computing a graph of instance dependencies. With the **indirect** exception of the **second example**, which is only one level away from its instance, this proposal is focused on that which is easy for all compilers to compute and verify.

The wish

If the `Simpler implicit move` [1:3] proposal does not fix the following examples than it would be beneficial if `C++` was enhanced to include these fixes, along similar lines.

Fix the pointer version of the **first example**.

```
int* h(bool b, int i) {
  static int s;
  if (b) {
    return &s;  // OK
  } else {
    return &i;  // error: instance `i` dies before the returned pointer
  }
}
```

Just as the pointer or reference to a local should not be returned because it exits the scope of the lifetime of the local, this should also occur in the body of any given function.

```
void h(bool b, int i) {
  int* p = nullptr;
  if (b) {
    static int s;
    p = &s;  // OK
  } else {
    int i = 0;
    p = &i;  // error: instance `i` dies before pointer `p`
  }
  // ...
}
```

In order to address these types of dangling, in the language, we need to add a rule into the standard.

RULE: You can not directly assign the address of an instance to a pointer or a reference if the instance dies before the pointer or reference dies.

At worse, this is dangling. At best, this is a logic error.

I am not asking in this proposal for a resolution for a `std::optional<std::reference_wrapper<int>>` version of the last example. While that does need to be fixed, that would get into two additional levels of indirection. I am limiting this proposal just to one level of additional indirection, as the `Simpler implicit move` [1:4] proposal did. Any more levels of indirection would require compilers, at greater compilation cost, to process the instance dependency graph and then do more when programmers can contribute to the dependency information via an attribute that documents whether returns and parameters are dependent upon one another.

The next set of examples are related to the **second example** that has just one additional level of **indirect**ion. Just like `std::reference_wrapper` , lambda functions and coroutines that have pointers or references to locals should not be able to be returned from a function.

```
auto lambda() {
    int i = 0;
    return [&i]() -> &int
        {
            return i;
        };  // error: instance `i` dies before the returned lambda
}
```

```
auto coroutine() {
    int i = 0;
    return [&i]() -> generator<int>
        {
            co_return i;
        };  // error: instance `i` dies before the returned coroutine
}
```

Consequently, another rule needs to be added.

RULE: You can not return a lambda or coroutine that captures a pointer or reference to a local.

While a combination of these two rules would mean `std::optional` of a lambda or coroutine that captures a pointer or reference to a local should also be invalid, this proposal does not ask for that to be fixed, since it would get into two additional levels of indirection. I am limiting this proposal just to one additional level of indirection as the `Simpler implicit move` [1:5] proposal did. Any more levels of indirection would require compilers, at greater compilation cost, to process the instance dependency graph and then do more when programmers can contribute to the dependency information via an attribute that documents whether returns and parameters are dependent upon one another.

Summary

`c++` can fix more dangling simply in the language without the need for any additional non standard static analyzers. The cost is minimal but like all dangling, these defects are big. Enforcing these rules will improve the safety of `c++`. Further, the pointer versions of these safety checks could be contributed back to `c` thus making it safer and improving our ecosystem as a whole.

Frequently Asked Questions

Why not handle more indirect cases?

Compilers and the standard should. Forget the increased compile time, compilers should create and analyze the instance dependency graph to produce as many errors as possible to make `c++` much safer. Library authors should be able to contribute to this graph for more of the indirect cases. This can be done in three phases.

- direct dangling and low indirection level count dangling as exemplified by the `Simpler implicit move` [1:6] and this proposal
- maximum indirect dangling detection without library author contributions
- maximum indirect dangling detection with library author contributions

As the first phase is cheaper and easier to do than we should get this and similar proposals into the language as quickly as possible, as a triage measure. Later, the other phases can be addressed in turn whether in the same or succeeding releases.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html) ↩ ↩ ↩ ↩ ↩ ↩ ↩

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf) ↩

3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf) ↩

4. https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object) ↩ ↩ ↩